for math reasoning. In general, we found that the efficacy of a given approach heavily correlates with the difficulty of the problem from the perspective of the base LLM's capabilities. This motivated us to introduce the notion of "compute-optimal" scaling of test-time computation, which prescribes a adaptive, prompt-dependent strategy to improve performance under a given test-time compute budget. By applying such a compute-optimal scaling strategy, we find that can improve the efficiency of test-time compute scaling by a factor of $2 - 4\times$. When comparing benefits obtained from additional test-time compute against benefits from additional pre-training compute in a FLOPs-matched setting, we show for the first time that using test-time computation with seemingly simple methods (i.e., revisions and search) can already scale well on certain types of prompts, providing gains over spending those FLOPs in pretraining. That said, there are also limitations associated with our study that future work can aim to address.

Further improving test-time compute scaling. In this work we focused on improving the test-time compute scaling of two primary mechanisms: the verifier and the proposal distribution (via revisions). While we combined verifiers with revisions in Section 6, we did not experiment with PRM tree-search techniques in combination with revisions. Neither did we study other techniques such as critique and revise [23]. Future work should investigate how test-time compute scaling can be further improved by combining a variety of these approaches. Additionally, we found that across the board these schemes provided small gains on hard problems; future work should work to develop new ways of using test-time compute which can circumvent this limitation.

Assessing question difficulty quickly. We used a notion question difficulty as a simple sufficient statistic for approximating the compute-optimal test-time scaling strategy. While this scheme was effective, estimating our notion of difficulty requires applying a non-trivial amount of test-time compute itself. Future work should consider alternative ways of more efficiently estimating question difficulty (e.g., by pretraining or finetuning models to directly predict difficulty of a question) or dynamically switching between assessing difficulty and attempting to solve a question.

Interleaving test-time and training-time compute. We focused purely on test-time compute scaling in this work and the degree to which test-time compute can be traded off for additional pretraining. However, in the future, we envision that the outputs of applying additional test-time compute can be distilled back into the base LLM, enabling an iterative self-improvement loop that operates on open-ended natural language. To this end, future work should extend our findings and study how the outputs of applying test-time compute can be used to improve the base LLM itself.

Acknowledgements

We thank Yi Su, Rishabh Agarwal, Yinlam Chow, Aleksandra Faust, Vincent Zhuang, George Tucker, Hao Liu, Jiayi Pan, Ethan Dyer, Behnam Neyshabur, Xavier Garcia, Yamini Bansal, Lampros Lamprou, Yuxiao Qu, and Amrith Setlur for their feedback on an earlier version of the paper and discussions. We attribute and thank Rishabh Agarwal, Vincent Zhuang, Yi Su, and Avi Singh for ideas and discussions, and experiments that concretely demonstrated the promise of pairwise sample generation for training revision models, and edit distance based sampling in [1]. We thank Slav Petrov for leadership support.

References

[1] Training revision models with synthetic data. Coming soon, 2024.

[2] C. Andrieu, N. De Freitas, A. Doucet, and M. I. Jordan. An introduction to mcmc for machine learning. 2003.

[3] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, E. Chu, J. H. Clark, L. E. Shafey, Y. Huang, K. Meier-Hellstern, G. Mishra, E. Moreira, M. Omernick, K. Robinson, S. Ruder, Y. Tay, K. Xiao, Y. Xu, Y. Zhang, G. H. Abrego, J. Ahn, J. Austin, P. Barham, J. Botha, J. Bradbury, S. Brahma, K. Brooks, M. Catasta, Y. Cheng, C. Cherry, C. A. Choquette-Choo, A. Chowdhery, C. Crepy, S. Dave, M. Dehghani, S. Dev, J. Devlin, M. Díaz, N. Du, E. Dyer, V. Feinberg, F. Feng, V. Fienber, M. Freitag, X. Garcia, S. Gehrmann, L. Gonzalez, G. Gur-Ari, S. Hand, H. Hashemi, L. Hou, J. Howland, A. Hu, J. Hui, J. Hurwitz, M. Isard, A. Ittycheriah, M. Jagielski, W. Jia, K. Kenealy, M. Krikun, S. Kudugunta, C. Lan, K. Lee, B. Lee, E. Li, M. Li, W. Li, Y. Li, J. Li, H. Lim, H. Lin, Z. Liu, F. Liu, M. Maggioni, A. Mahendru, J. Maynez, V. Misra, M. Moussalem, Z. Nado, J. Nham, E. Ni, A. Nystrom, A. Parrish, M. Pellat, M. Polacek, A. Polozov, R. Pope, S. Qiao, E. Reif, B. Richter, P. Riley, A. C. Ros, A. Roy, B. Saeta, R. Samuel, R. Shelby, A. Slone, D. Smilkov, D. R. So, D. Sohn, S. Tokumine, D. Valter, V. Vasudevan, K. Vodrahalli, X. Wang, P. Wang, Z. Wang, T. Wang, J. Wieting, Y. Wu, K. Xu, Y. Xu, L. Xue, P. Yin, J. Yu, Q. Zhang, S. Zheng, C. Zheng, W. Zhou, D. Zhou, S. Petrov, and Y. Wu. Palm 2 technical report, 2023.

[4] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, C. Chen, C. Olsson, C. Olah, D. Hernandez, D. Drain, D. Ganguli, D. Li, E. Tran-Johnson, E. Perez, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, K. Lukosuite, L. Lovitt, M. Sellitto, N. Elhage, N. Schiefer, N. Mercado, N. DasSarma, R. Lasenby, R. Larson, S. Ringer, S. Johnston, S. Kravec, S. E. Showk, S. Fort, T. Lanham, T. Telleen-Lawton, T. Conerly, T. Henighan, T. Hume, S. R. Bowman, Z. Hatfield-Dodds, B. Mann, D. Amodei, N. Joseph, S. McCandlish, T. Brown, and J. Kaplan. Constitutional ai: Harmlessness from ai feedback, 2022.

[5] C. Blakeney, M. Paul, B. W. Larsen, S. Owen, and J. Frankle. Does your data spark joy? performance gains from domain upsampling at the end of training, 2024. URL https://arxiv.org/abs/2406.03476.

[6] G. Chen, M. Liao, C. Li, and K. Fan. Alphamath almost zero: process supervision without process, 2024.

[7] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems, 2021.

[8] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023.

[9] J. S. B. T. Evans. Heuristic and analytic processes in reasoning. *British Journal of Psychology*, 75(4): 451–468, 1984.

[10] X. Feng, Z. Wan, M. Wen, S. M. McAleer, Y. Wen, W. Zhang, and J. Wang. Alphazero-like tree-search can guide large language model decoding and training, 2024.

[11] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. Pal: Program-aided language models, 2023. URL https://arxiv.org/abs/2211.10435.

[12] S. Goyal, Z. Ji, A. S. Rawat, A. K. Menon, S. Kumar, and V. Nagarajan. Think before you speak: Training language models with pause tokens, 2024. URL https://arxiv.org/abs/2310.02226.

[13] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset, 2021.

[14] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models, 2022.

[15] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou. Large language models cannot self-correct reasoning yet, 2023.

[16] A. L. Jones. Scaling scaling laws with board games, 2021. URL https://arxiv.org/abs/2104.03113.

[17] D. Kahneman. Maps of bounded rationality: Psychology for behavioral economics. *The American Economic Review*, 93(5):1449–1475, 2003.

[18] D. Kahneman. *Thinking, fast and slow*. Farrar, Straus and Giroux, New York, first paperback edition edition, 2013.

[19] L. Kocsis and C. Szepesv'ari. Bandit based monte-carlo planning. In *European conference on machine learning*, pages 282–293. Springer, 2006.

[20] A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo, Y. Wu, B. Neyshabur, G. Gur-Ari, and V. Misra. Solving quantitative reasoning problems with language models, 2022.

[21] Y. Li, Z. Lin, S. Zhang, Q. Fu, B. Chen, J.-G. Lou, and W. Chen. Making large language models better reasoners with step-aware verifier, 2023.

[22] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step, 2023.

[23] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark. Self-refine: Iterative refinement with self-feedback, 2023.

[24] N. McAleese, R. Pokorny, J. F. Cerón Uribe, E. Nitishinskaya, M. Trębacz, and J. Leike. Llm critics help catch llm bugs. *OpenAI*, 2024.

[25] OpenAI. Gpt-4 technical report, 2024.

[26] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL https://arxiv.org/abs/2307.16789.

[27] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, and J.-R. Wen. Tool learning with large language models: A survey, 2024. URL https://arxiv.org/abs/2405.17935.

[28] Y. Qu, T. Zhang, N. Garg, and A. Kumar. Recursive introspection: Teaching foundation models how to self-improve. 2024.